

PANTHEON

PERSONAL AGI CIVILIZATION

A self-organizing multi-agent civilization that thinks, builds, and evolves — running entirely on an M4 MacBook.

THE CLAIM

A society of small identical transformer agents, given only a hard problem and a shared substrate, will self-organize into specialized roles, form economic coalitions, produce generalist artifacts (code, research, visuals), and distill collective knowledge — without any hardcoded role assignment. Emergent specialization is sufficient for AGI-level collective problem solving at laptop scale.

#	PAPER	ROLE IN PANTHEON
01	Generative Agents — Park et al., Stanford 2023	Agent memory + behavior architecture
02	CAMEL — Li et al., 2023	Multi-agent communication protocol
03	nanoGPT — Karpathy 2022	Your agent transformer base
04	NOTEARS — Zheng et al., 2018	Causal discovery in coalition layer
05	Diffuser — Janner et al., 2022	Trajectory planning for agent goals
06	Voyager — Wang et al., 2023	Skill library + lifelong learning pattern
07	EvoPrompting — Chen et al., 2023	Evolutionary pressure on agent strategies

WK	COMPONENT	GPU?	SUCCESS METRIC
01	Substrate + Economy	No	10 agents alive, economy stable
02	Agent Core Transformer	No*	Agent predicts next token, loss < 3.0
03	Speciation Engine	No	3+ distinct role clusters emerge
04	Coalition Formation	No	Coalition solves task solo agent cannot

05	Generative Output Layer	Yes	Code runs, image generates, hypothesis coherent
06	Shared Memory + Distillation	No	Knowledge graph grows, recall > 80%
07	Self-Modeling + Evolution	No	Society improves on repeated problem class
08	Historian + Meta-Output	No	Society writes its own paper draft
09	Dashboard + Visualization	No	Live D3 graph of agent society
10	Demo + Hardening	No	90-second demo, reproducible from scratch

ARCHITECTURE OVERVIEW

PANTHEON is built bottom-up. Every layer depends on the one below. Do not build the Coalition layer before Speciation. Do not build Speciation before the Agent Core. The society is useless without agents. Agents are useless without a substrate.

LAYER	COMPONENT	DEPENDS ON
0 — Foundation	Substrate + Economy	Nothing. Build first.
1 — Cognition	Agent Core Transformer	Substrate
2 — Divergence	Speciation Engine	Agent Core + Substrate
3 — Cooperation	Coalition Formation	Speciation + Substrate
4 — Production	Generative Output Layer	Coalition + Agent Core
5 — Persistence	Shared Memory + Distillation	All above
6 — Recursion	Self-Modeling + Evolution	All above
7 — Narrative	Historian + Meta-Output	Self-Modeling
8 — Interface	Dashboard + Visualization	All above

THE CORE INSIGHT

Every multi-agent system today assigns fixed roles: planner → coder → critic. That is a pipeline, not intelligence. PANTHEON's agents start **identical**. Roles emerge through three forces: (1) task exposure — agents that repeatedly succeed at code tasks develop a code-biased skill fingerprint; (2) reputation — other agents preferentially route tasks to high-reputation specialists; (3) economic pressure — agents that fail to earn compute tokens die and are replaced by offspring of successful agents. This is evolutionary pressure applied to cognition. It has never been done cleanly at small scale.

WHAT MAKES THIS AGI-LEVEL

PROPERTY	PANTHEON	CURRENT SOTA
Role assignment	Emergent from task exposure	Hardcoded strings
Memory	Shared + private + episodic	Context window only
Agent lifecycle	Birth → speciation → death	Static, never dies
Coordination	Market economy (compute tokens)	Fixed pipelines

Self-reflection	Society models itself	None
Output	Code + research + visuals	Usually text only
Learning signal	Society improves from own history	None between runs

WEEKS 1 – 2

WEEK

1

Substrate + Economy

The world all agents live in. Build this first. Everything else depends on it.

→

DELIVERABLE

10 agents spawned, economy stable for 1000 ticks. No deadlock, no starvation.

WHAT YOU BUILD

- SharedMemory: FAISS flat index (L2) + SQLite for episodic log. Runs entirely on CPU.
- ComputeToken economy: each agent starts with 100 tokens. Solving a task earns tokens. Spawning a sub-agent costs 20. Dying = balance < 0.
- CommunicationBus: simple pub/sub using Python asyncio.Queue. Agents post tagged Signal objects. Subscribers filter by tag.
- ArtifactStore: flat file store + SQLite index. Code, images, hypotheses stored with author_agent_id + timestamp.
- AgentRegistry: tracks all living agents, their skill fingerprints, reputation scores, token balances.
- TaskQueue: priority queue. Tasks have type (code/research/visual), difficulty score, reward tokens.

KEY CODE SHAPE

```
# substrate/economy.py

class ComputeEconomy:

    def __init__(self, n_agents, start_tokens=100):

        self.balances = {i: start_tokens for i in range(n_agents)}

    def earn(self, agent_id, reward): self.balances[agent_id] += reward

    def spend(self, agent_id, cost):

        self.balances[agent_id] -= cost

        if self.balances[agent_id] < 0: self.kill(agent_id)

    def kill(self, agent_id): # replace with offspring of top earner

        top = max(self.balances, key=self.balances.get)

        self.balances[agent_id] = self.balances[top] // 2

        return {'killed': agent_id, 'parent': top}

# substrate/memory.py

class SharedMemory:

    def __init__(self, dim=128):
```

```

import faiss

self.index = faiss.IndexFlatL2(dim)

self.store = {} # id -> (vector, metadata)

def write(self, vec, meta): # any agent can write
    idx = self.index.ntotal
    self.index.add(vec.reshape(1, -1))
    self.store[idx] = meta

def query(self, vec, k=5): # retrieve k nearest
    D, I = self.index.search(vec.reshape(1, -1), k)
    return [self.store[i] for i in I[0] if i in self.store]

```

CRITICAL DECISIONS

- Use asyncio throughout — agents are coroutines, not threads. Avoids GIL hell on M4.
- FAISS IndexFlatL2 is exact and fast enough for <100k vectors on M4. No GPU needed.
- Token economy must have a floor — agents never die permanently in Week 1. Disable death until Week 3.
- Test: run 10 agents for 1000 ticks. Plot token balances. You should see divergence, not flat lines. If flat: your reward signals are broken.

WEEK
2

Agent Core Transformer

Write your own. Every line. ~400 lines. This is the brain of each agent.

→

DELIVERABLE

Single agent predicts next action token. Val loss < 3.0. Runs on CPU in <200ms.

WHAT YOU BUILD

- GPT-2 nano: 6 layers, 6 heads, d_model=192, FFN 4x. ~3M params per agent. 10 agents = 30M total.
- RoPE positional embeddings — handles variable context, no learned positions.
- RMSNorm instead of LayerNorm — faster on M4, identical quality.
- Separate token embeddings for: task tokens, action tokens, memory tokens, special tokens.
- Private latent state: a 128-dim vector stored outside the transformer. Persists across forward passes.
- Context window: 512 tokens. Enough for task + memory retrievals + action history.
- Train on synthetic task traces first (generated, not real). Real tasks come in Week 4.

KEY CODE SHAPE

```
# agent/transformer.py — write every line yourself
```

```

class AgentTransformer(nn.Module):
def __init__(self, vocab=512, d=192, heads=6, layers=6, ctx=512):
super().__init__()
self.tok_emb = nn.Embedding(vocab, d)
self.type_emb = nn.Embedding(4, d) # task/action/memory/special
self.blocks = nn.ModuleList([Block(d, heads) for _ in range(layers)])
self.norm = RMSNorm(d)
self.lm_head = nn.Linear(d, vocab, bias=False)
self.private = nn.Parameter(torch.zeros(128)) # private latent

def forward(self, idx, tok_types):
x = self.tok_emb(idx) + self.type_emb(tok_types)
x = apply_rope(x) # rotary position embedding
for block in self.blocks: x = block(x)
return self.lm_head(self.norm(x))

class Block(nn.Module):
def forward(self, x):
x = x + self.attn(self.norm1(x)) # pre-norm residual
x = x + self.ffn(self.norm2(x))
return x

```

CRITICAL DECISIONS

- 3M params per agent is intentional. Intelligence comes from interactions, not individual model size.
- Private latent `self.private` is the agent's 'self'. It drifts via gradient updates — this is how speciation begins.
- Use `F.scaled_dot_product_attention` (PyTorch 2.x built-in). Free flash-attention equivalent on M4.
- Do NOT use KV cache yet. Add it in Week 6 when inference speed matters for the dashboard.
- Sanity check: overfit one agent on 100 synthetic traces. Loss should reach 0.1. If not, your data pipeline is broken.

WEEKS 3 – 4

WEEK

3

Speciation Engine

Agents start identical. This week, they diverge. Roles emerge. Nobody assigns them.



DELIVERABLE

After 500 tasks: t-SNE of skill fingerprints shows 3+ distinct clusters with no label supervision.

WHAT YOU BUILD

- SkillFingerprint: 128-dim vector per agent. Initialized identically. Updated after every task.
- Update rule: if agent succeeds at a code task, nudge fingerprint toward the 'code attractor' — a learned centroid in skill space.
- Attractors are NOT predefined. They emerge as the mean of successful agents in each task type.
- Reputation system: after task completion, 3 random agents score the output (0-1). Score updates solver's reputation.
- Task routing: new tasks are routed to the agent with highest dot product between task embedding and skill fingerprint.
- Death + inheritance: agents below 0 tokens die. Offspring inherits parent's skill fingerprint + 50% noise. Evolution.
- Visualization: t-SNE of all agent fingerprints every 100 ticks. Watch clusters form in real time.

KEY CODE SHAPE

```
# speciation/engine.py

class SpeciationEngine:

    def update_fingerprint(self, agent, task_type, success, reward):

        if not success: return

        attractor = self.get_attractor(task_type) # mean of winners

        alpha = 0.05 * reward # stronger pull for bigger wins

        agent.fingerprint += alpha * (attractor - agent.fingerprint)

        agent.fingerprint /= agent.fingerprint.norm() # keep on unit sphere

    def get_attractor(self, task_type):

        winners = [a.fingerprint for a in self.agents

        if a.task_history[-1] == (task_type, True)]

        if not winners: return torch.randn(128) # cold start

        return torch.stack(winners).mean(0)
```



```
def route_task(self, task):
    task_emb = self.task_encoder(task) # small MLP, 128-dim
    scores = [torch.dot(task_emb, a.fingerprint) for a in self.agents]
    return self.agents[torch.tensor(scores).argmax()]
```

CRITICAL DECISIONS

- The task encoder (task → 128-dim) is a tiny 2-layer MLP. Train it jointly with the fingerprint update. ~10k params.
- Expected clusters after 500 tasks: code-solvers, research-generators, creative/visual agents. You did NOT define these.
- If clusters don't form after 500 tasks: your reward signal is too uniform. Add difficulty tiers to tasks.
- Log every fingerprint update. You'll use this for the Week 8 latent space visualization.
- Mutation rate on inheritance: $0.1 * \text{torch.randn}(128)$. Too low = no diversity. Too high = no learning.

WEEK
4

Coalition Formation

Hard problems require teams. Agents bid, win, self-organize, elect a coordinator.

→

DELIVERABLE

A coalition of 3–5 agents solves a task that any single agent fails alone. Measurably.

WHAT YOU BUILD

- TaskDecomposer: given a hard task, the highest-reputation agent decomposes it into subtasks (LLM call or rule-based for now).
- Auction mechanism: subtasks are posted to the bus. Agents bid compute tokens based on skill match score.
- Coalition contract: winning bidders sign a contract — they pool token rewards, share a communication channel.
- Coordinator election: agent with highest reputation in the coalition gets read-access to all members' private latents.
- Coordinator can issue directives — other agents weight their next-token distribution toward directive tokens.
- Result aggregation: each agent produces a partial output. Coordinator merges using a small learned aggregator.
- Reputation update after coalition task: individual scores by quality of their subtask output.

KEY CODE SHAPE

```
# coalition/formation.py

class CoalitionFormation:

    def form(self, hard_task):
```

```

subtasks = self.decompose(hard_task) # list of simpler tasks

coalition = []

for subtask in subtasks:

    bids = self.run_auction(subtask) # agents bid tokens

    winner = max(bids, key=lambda b: b.score - b.cost)

    self.economy.spend(winner.agent_id, winner.cost)

    coalition.append((winner.agent_id, subtask))

    coordinator = self.select_coordinator(coalition)

return Coalition(members=coalition, coordinator=coordinator)

```

```

def run_auction(self, subtask):

    task_emb = self.encode(subtask)

    bids = []

    for agent in self.agents:

        skill_match = torch.dot(task_emb, agent.fingerprint).item()

        bid_cost = max(1, int(10 * (1 - skill_match))) # specialists bid cheap

        bids.append(Bid(agent.id, score=skill_match, cost=bid_cost))

    return bids

```

CRITICAL DECISIONS

- Specialists bid cheap for their domain tasks. Generalists bid expensive. This naturally routes tasks correctly.
- Coalition size: minimum 2, maximum 7. Hard cap prevents one agent from winning all auctions.
- Coordinator's latent read-access is the key mechanism — it can see what each member 'knows'. Novel mechanism.
- Test coalitions against solo agents on the same hard task. Coalition must win >60% of the time. If not: your decomposition is bad.
- Log every coalition: who formed it, who won what subtask, final score. This data feeds Week 7 self-modeling.

WEEKS 5 – 6

WEEK

5

Generative Output Layer

The civilization now produces real artifacts. Code that runs. Research that's coherent. Visuals that exist.

→

DELIVERABLE

Given one hard problem, PANTHEON produces: working code, a research hypothesis, and a visual. All from one run.

WHAT YOU BUILD

- CodeAgent output: agent emits code tokens → sandboxed Python execution (subprocess, 5s timeout) → execution result fed back as reward.
- ResearchAgent output: structured hypothesis in JSON: {claim, evidence_needed, experiment_design, falsifiability_condition}.
- VisualAgent output: agent emits a DiT (Diffusion Transformer) prompt + style vector → call a local tiny diffusion model or SDXL API.
- OutputRouter: coordinator decides which coalition members produce which output type based on their specialization.
- Quality scorer: each output type has a domain-specific scorer (code: does it run + pass tests; research: coherence score via small LM; visual: CLIP score).
- ArtifactStore integration: every output stored with agent_id, coalition_id, task_id, quality_score, timestamp.
- Cross-pollination: code agent's output can be referenced by research agent as evidence. Artifacts link to each other.

KEY CODE SHAPE

```
# output/code_agent.py

class CodeOutputLayer:

    def execute(self, agent, task):

        code_tokens = agent.generate(task, max_tokens=256)

        code_str = self.detokenize(code_tokens)

        result = self.sandbox(code_str, timeout=5) # subprocess

        reward = 1.0 if result.success else -0.3

        self.economy.earn(agent.id, reward * 10)

        return Artifact(type='code', content=code_str,

            result=result, author=agent.id)

# output/research_agent.py
```

```

class ResearchOutputLayer:
    SCHEMA = {
        'claim': str, # testable statement
        'evidence_needed': list,
        'experiment': str, # how to test it
        'falsifiable': bool, # Popperian check
    }

    def generate(self, agent, task):
        raw = agent.generate(task, max_tokens=128)
        hyp = self.parse_to_schema(raw, self.SCHEMA)
        score = self.coherence_score(hyp) # small LM judge
        self.economy.earn(agent.id, score * 15)
        return Artifact(type='research', content=hyp, author=agent.id)

```

CRITICAL DECISIONS

- Sandbox code execution with subprocess + resource limits: CPU time=5s, memory=256MB. Never exec() directly.
- For visuals on M4: use a quantized SDXL-turbo (4-step) locally via diffusers. Or call Stability API. Either works.
- Cross-pollination is the magic: a research hypothesis that references a code artifact is scored higher. Incentivize linking.
- Week 5 goal is breadth, not depth. All three output types must work end-to-end, even if quality is mediocre.
- Quality scorer for research: prompt a small local LLM (Llama-3.2-1B) to rate coherence 0-1. That's your reward signal.

WEEK

6

Shared Memory + Knowledge Distillation

The society gets smarter over time. Without retraining. This is how.

→

DELIVERABLE

After 1000 tasks, knowledge graph has >200 nodes. Recall of past solutions > 80% on seen problem types.

WHAT YOU BUILD

- EpisodicMemory: every completed task stored as (task_embedding, solution_embedding, quality_score, agent_coalition). FAISS-indexed.

- SemanticMemory: compressed knowledge graph — nodes are concepts, edges are learned relationships. Built by the Librarian agent.
- LibrarianAgent: emergent specialist (should speciate naturally). Runs every 100 ticks. Reads episodic memory, compresses into semantic nodes.
- Compression: k-means on solution embeddings → cluster centers become semantic nodes → label each cluster with top tokens.
- Retrieval-augmented generation: before any agent tackles a task, it queries shared memory for similar past tasks + solutions.
- Knowledge graph stored as NetworkX graph + SQLite. Nodes have embedding + label + frequency. Edges have weight + type.
- Forgetting: solutions with quality < 0.3 decay. High-quality solutions get stronger connections. Hebbian rule.

KEY CODE SHAPE

```
# memory/distillation.py

class KnowledgeDistiller:

def distill(self, episodic_memory, threshold=100):

if len(episodic_memory) < threshold: return

vecs = torch.stack([m.solution_emb for m in episodic_memory])

labels = KMeans(n_clusters=20).fit_predict(vecs.numpy())

for cluster_id in range(20):

members = [m for m,l in zip(episodic_memory, labels)

if l == cluster_id]

centroid = vecs[labels==cluster_id].mean(0)

label = self.top_tokens(members) # most common tokens

quality = np.mean([m.quality for m in members])

self.knowledge_graph.add_node(

cluster_id, embedding=centroid,

label=label, quality=quality)

self.add_edges() # cosine similarity > 0.7 → edge

def retrieve(self, task_emb, k=3):

D, I = self.faiss_index.search(task_emb, k)

return [self.knowledge_graph.nodes[i] for i in I[0]]
```

CRITICAL DECISIONS

- LibrarianAgent should emerge naturally from speciation (Week 3). If it hasn't: manually seed one agent with high memory-task reward.
- Distillation runs every 100 ticks on a background thread. Never block the main agent loop.

- Hebbian forgetting: every tick, decay all edge weights by 0.001. Strengthen by 0.1 when a connected node is retrieved.
- RAG at inference: prepend top-3 retrieved solutions as context tokens to agent's input. This is where the society gets smarter.
- Test: give the society a problem it solved before (with noise). Solution quality should be >20% better than first attempt.

WEEKS 7 – 8

WEEK

7

Self-Modeling + Evolution

The society watches itself. Learns from its own behavior. Gets better without retraining.

→

DELIVERABLE

Society performance on a benchmark problem class improves >30% between run 1 and run 5 of the same class.

WHAT YOU BUILD

- HistorianAgent: specialist that reads all logs — coalition formations, task outcomes, agent deaths, fingerprint trajectories.
- SocietyModel: a small transformer (same architecture as agent core) trained on society event sequences. Predicts: which coalition will succeed, which agent will die next.
- Meta-reward: agents that behave as the SocietyModel predicts get a small bonus. Agents that surprise it get a curiosity bonus.
- Evolutionary pressure: every 200 ticks, bottom 10% of agents (by token balance) die. Their offspring inherit from top 10% + mutation.
- Strategy mutation: offspring don't just inherit fingerprints — they inherit bidding strategies with small random perturbations.
- Benchmark harness: a fixed set of 20 hard problems, run every 500 ticks. Plot society score over time. This is your main result graph.
- Self-report: HistorianAgent generates a structured log every 100 ticks: {top_agents, emerging_roles, bottlenecks, recommendations}.

KEY CODE SHAPE

```
# evolution/self_model.py

class SocietyModel(nn.Module): # same arch as AgentTransformer

# Trained on: sequences of society events

# Event types: TASK_POSTED, BID_WON, COALITION_FORMED,
# TASK_SUCCESS, TASK_FAIL, AGENT_DIED, AGENT_BORN

# Predicts: next event type + outcome probability

pass


class EvolutionEngine:

def evolve(self, agents, economy, tick):

if tick % 200 != 0: return
```

```

ranked = sorted(agents, key=lambda a: economy.balances[a.id])
bottom = ranked[:len(ranked)//10] # bottom 10%
top = ranked[-len(ranked)//10:] # top 10%
for dead in bottom:
    parent = random.choice(top)
    child = self.spawn(parent, mutation_rate=0.1)
    agents.remove(dead)
    agents.append(child)
    economy.balances[child.id] = economy.balances[parent.id] // 2

def spawn(self, parent, mutation_rate):
    child = Agent()
    child.fingerprint = parent.fingerprint + mutation_rate * torch.randn(128)
    child.fingerprint /= child.fingerprint.norm()
    child.strategy = parent.strategy.copy() # bidding params
    child.strategy += mutation_rate * np.random.randn(len(child.strategy))
    return child

```

CRITICAL DECISIONS

- SocietyModel trains online — it sees each new event as it happens. Use a sliding window of last 1000 events.
- Curiosity bonus for surprising the SocietyModel: agents that do unexpected things get +5 tokens. Prevents local optima.
- Evolution tick=200 is aggressive. If society destabilizes (all agents dying): increase to 500.
- Benchmark problems must be fixed across all runs. Randomize order, not the problems themselves.
- The self-report from HistorianAgent becomes your paper's qualitative results section. Save every one.

WEEK

8

Historian + Meta-Output

The society writes a paper about itself. This is your publishable result and your demo anchor.

→

DELIVERABLE

A structured research report generated entirely by PANTHEON agents, describing their own behavior. Coherent enough to submit.

WHAT YOU BUILD

- HistorianAgent now has write access to a structured paper template: {abstract, introduction, methods, results, discussion}.
- Abstract: synthesized from the top-level SocietyModel self-reports. HistorianAgent compresses 10 reports into 200 words.
- Methods: auto-generated from the coalition formation logs — which agents formed, what strategies emerged, how tasks were routed.
- Results: the benchmark improvement graph (Week 7) + t-SNE of role clusters (Week 3) + knowledge graph growth curve (Week 6).
- Discussion: HistorianAgent runs a structured debate — 3 agents argue about why the society succeeded or failed. Majority view included.
- All writing done via Llama-3.2-3B-Instruct running locally on M4. HistorianAgent provides structured prompts, not free-form.
- Output: a PDF (via reportlab) + a JSON structured version for downstream use.

KEY CODE SHAPE

```
# historian/meta_output.py

class HistorianAgent:

    SECTIONS = ['abstract', 'introduction', 'methods', 'results', 'discussion']

    def write_paper(self, society_logs, benchmark_results):

        paper = {}

        for section in self.SECTIONS:

            prompt = self.build_prompt(section, society_logs, benchmark_results)

            response = self.llm.generate(prompt, max_tokens=400)

            paper[section] = response

        paper['figures'] = self.collect_figures() # t-SNE, benchmark curve

        return paper

    def structured_debate(self, topic, n_agents=3):

        # pick 3 agents with highest reputation

        debaters = sorted(self.agents, key=lambda a: a.reputation)[-3:]

        arguments = [a.generate(f'Argue about: {topic}') for a in debaters]

        # vote on most coherent argument

        winner = self.vote(arguments) # agents score each other

        return arguments, winner
```

CRITICAL DECISIONS



Llama-3.2-3B-Instruct runs on M4 in ~4GB RAM via llama.cpp or MLX. Use MLX for M4 — 3x faster than llama.cpp.

- HistorianAgent must be the highest-reputation agent in the society. If it isn't: add a prestige bonus for distillation tasks.
- The structured debate is your most novel qualitative result. Three AI agents arguing about their own society's behavior. Record it.
- Paper quality bar: coherent, factually grounded in the actual logs, no hallucinations about events that didn't happen. Verify every claim.
- This week's output IS your conference submission abstract. Write it to that standard from the start.

WEEKS 9 – 10

WEEK

9

Dashboard + Visualization

Make the invisible visible. The society is your demo. The dashboard is your stage.



DELIVERABLE

A live web dashboard showing PANTHEON running in real time. Agent nodes, coalitions, economy, artifacts. All live.

WHAT YOU BUILD

- FastAPI backend: WebSocket endpoint streams society events at 10Hz. REST endpoints for artifact retrieval, history, benchmark scores.
- React frontend: D3.js force graph — nodes are agents (size=reputation, color=specialization cluster), edges=active coalitions.
- Economy panel: live bar chart of token balances. Watch agents rise and fall in real time.
- Artifact feed: scrolling list of latest artifacts (code, research, visuals) with quality scores and author agent IDs.
- Latent space view: 2D t-SNE projection of all agent fingerprints, updated every 30s. Click a cluster to see its members' tasks.
- Knowledge graph view: NetworkX graph rendered in D3. Click a node to see all tasks that contributed to it.
- Replay mode: given a task ID, replay the entire coalition formation and execution as an animation.

KEY CODE SHAPE

```
# dashboard/backend.py

from fastapi import FastAPI, WebSocket
import asyncio, json

app = FastAPI()

@app.websocket('/ws/society')
async def society_stream(ws: WebSocket):
    await ws.accept()
    while True:
        snapshot = {
            'agents': [a.to_dict() for a in pantheon.agents],
            'coalitions': [c.to_dict() for c in pantheon.active_coalitions],
            'economy': pantheon.economy.balances,
            'tick': pantheon.tick,
```

```
'latest_artifacts': pantheon.artifact_store.recent(5),
}

await ws.send_text(json.dumps(snapshot))

await asyncio.sleep(0.1) # 10Hz

# frontend/AgentGraph.jsx (D3 force simulation)

// nodes: agents | edges: active coalitions

// node radius → reputation score

// node color → speciation cluster (from k-means, Week 3)

// edge width → coalition strength (token pool size)
```

CRITICAL DECISIONS

- D3 force simulation with ~50 nodes runs at 60fps on any laptop. Do not use WebGL unless you have >500 agents.
- Color-code agents by their speciation cluster. Use the k-means labels from Week 3. 4 colors max for readability.
- The replay mode is your killer demo feature. Record a 30-second coalition solving a hard problem. That's your tweet.
- Dashboard must work offline — bundle everything. Your demo will be in a conference room with bad wifi.
- Add a 'stress test' button: floods society with 50 tasks at once. Watch coalition formation under pressure. Spectacular to watch.

WEEK
10

Demo + Hardening

If it doesn't look good in 90 seconds, keep iterating. This week is about the story.

→

DELIVERABLE

**A 90-second screen recording that makes an AI researcher stop scrolling.
Reproducible from a single script.**

WHAT YOU BUILD

- Demo script: one command launches PANTHEON, feeds it a hard problem, runs for 60 seconds, shows coalition form and solve it.
- Hard problem for demo: 'Build a web scraper, write a research hypothesis about what you'll find, and generate a diagram of the data flow.'
- This requires: code agent + research agent + visual agent, in a coalition, with the coordinator merging outputs. All live.

- Reproducibility: docker-compose up or a single install.sh. Anyone with an M4 should be able to run it in <10 minutes.
- README: written by PANTHEON's HistorianAgent (Week 8 output), edited by you. Make this a point in the paper.
- Paper abstract: 200 words. Claim: emergent specialization + economic coordination = collective AGI behavior at laptop scale.
- Submit to: arXiv (cs.AI + cs.MA). Post on Twitter/X with the replay video. Tag Karpathy, tag LeCun. You earned it.

KEY CODE SHAPE

```
# run_demo.py – the only file anyone will run

import pantheon

society = pantheon.Society(n_agents=20, seed=42)

society.start()

hard_problem = '''
Build a Python web scraper for Hacker News front page.
Write a hypothesis about what topics dominate today.
Generate a data flow diagram of the scraper architecture.
'''

result = society.solve(hard_problem, timeout=60)

print('=== PANTHEON RESULT ===')
print('Code artifact: ', result.code.path)
print('Research artifact:', result.research.hypothesis)
print('Visual artifact: ', result.visual.path)
print('Coalition formed: ', result.coalition.member_ids)
print('Ticks elapsed: ', result.ticks)
print('Economy snapshot: ', result.economy_snapshot)

society.dashboard.open() # opens browser, live view
```

CRITICAL DECISIONS

- The demo problem is carefully chosen: it requires all three output types, a coalition, and a coordinator. Every feature is exercised.
- seed=42 in your demo — reproducible results are mandatory. Show the same run twice. Both must match.
- 90 seconds is your constraint. Practice the screen recording 10 times. Cut ruthlessly.

- The fact that the README was partially written by the society itself is a meta-result. Include a footnote saying so.
- One sentence pitch: 'We gave 20 identical agents a hard problem and a shared economy. They organized themselves, specialized, and solved it.' Done.

THE PHILOSOPHY

PANTHEON is not a framework. It is not a prompt wrapper. It is not 'agents' in the LangChain sense. It is a computational society — a system where intelligence is a property of the collective, not of any individual component.

The M4 is not a limitation. It is the constraint that forces the real insight: that collective intelligence at scale does not require large models. It requires correct incentives, correct memory, and correct evolutionary pressure. A society of 3M-parameter agents with a well-designed economy will outperform a single 70B model on tasks that require coordination, specialization, and memory.

WHAT SHOCKS PEOPLE

CLAIM	WHY IT'S SURPRISING
Roles emerge — nobody assigns them	Every MAS paper hardcodes roles. You don't.
Economy = coordination mechanism	Nobody uses token economies for agent routing at this scale.
Society writes its own paper	The meta-output is the result. Recursive and legit.
Runs on M4	People assume AGI-level behavior needs datacenter compute.
3M params per agent is enough	Individual scale doesn't matter. Collective structure does.
Reproducible in <10 min	Most MAS papers are not reproducible at all.

THE ONE-SENTENCE PITCH

"We gave 20 identical agents a hard problem and a shared economy. They organized themselves into specialists, formed coalitions, produced code, research, and visuals — and wrote a paper about themselves. It runs on a MacBook."

THE DISCIPLINE

Every week ends with a quantitative result and a visualization. Not 'I implemented X.' But 'X achieves Y metric, here is what it looks like.' A society that can't be measured can't be improved. A result that can't be visualized can't be communicated. Both are required. Always.